Savitribai Phule Pune University

Army Institute of Technology, Pune

Alandi Rd, Dighi, Pune – 411015

DEPARTMENT OF COMPUTER ENGINEERING

WEB TECHNOLOGY LABORATORY

MINI PROJECT

SwiftAd- Decentralized Advertisement Platform

ACADEMIC YEAR 2025-2026

TE COMP – B

Submitted By-

NAME	ROLL NUMBER
1. Ashish Kumar	7315
2. Nishant Singh	7339

Submitted to – Prof. Asha P Sathe

Savitribai Phule Pune University

Army Institute of Technology, Pune

Alandi Rd, Dighi, Pune – 411015

DEPARTMENT OF COMPUTER ENGINEERING

Onward to Glory

Certificate

This is to certify that the Project is entitled

SwiftAd- Decentralized Advertisement Platform

has been completed by

Ashish Kumar (7315) and Nishant Singh (7339)

of TE COMP -B in the Semester – 2 of academic year 2025-2026 in partial fulfilment of the Third Year of bachelor's degree in "Computer Engineering" as prescribed by the Savitribai Phule Pune University.

Prof. Asha P Sathe	Dr. Sunil Dhore
Project Guide	Head of Department

Place: AIT, Pune

Date: / /2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our subject in-charge, **Prof. Asha P Sathe**, for her valuable guidance, continuous support, and encouragement throughout the development of this project. Her insightful suggestions, timely feedback, and constant motivation played a significant role in helping us understand the concepts clearly and implement them effectively.

We are grateful to all those who have directly or indirectly contributed to the successful completion of this project. The process of developing this system required proper direction, consistent effort, and cooperation, and we feel fortunate to have received support from various sources during this journey.

We would also like to extend our sincere thanks to the Head of the Computer Department, **Dr. Sunil Dhore**, for providing us with the necessary infrastructure, resources, and a conducive environment for carrying out this project work successfully.

We are thankful to our classmates and friends for their valuable suggestions, constructive feedback, and continuous support during the development phase. Their cooperation helped us overcome various challenges and improve the quality of our project.

Ashish Kumar (7315)

Nishant Singh (7339)

T.E. Computer - B

Abstract

SwiftAd is a decentralized advertising platform designed to transform the digital advertising ecosystem by eliminating centralized intermediaries. Traditional advertising platforms retain 30–50% of publisher revenue, impose complex approval processes, and offer no transparency into payment flows. SwiftAd leverages blockchain technology, smart contracts, and decentralized storage to directly connect advertisers and publishers.

The platform enables advertisers to purchase ad space directly from publishers using the x402 micropayment protocol on the Stellar blockchain. Payments are settled instantly through Soroban smart contracts, and ad creatives are stored permanently on IPFS via the Lighthouse protocol. Publishers integrate SwiftAd into their websites using a lightweight JavaScript SDK that requires fewer than five minutes to set up.

A key innovation of the platform is its support for agentic ad placement, where AI agents can autonomously discover, bid on, and manage ad inventory on-chain using machine-readable payment headers — making SwiftAd one of the first advertising platforms natively designed for the agent economy.

The system is built using Next.js 14 for the full-stack framework, PostgreSQL with Prisma ORM for relational data management, Stellar and Soroban for blockchain operations, Freighter Wallet for non-custodial signing, and Lighthouse/IPFS for decentralized media storage. Security mechanisms such as NextAuth.js JWT authentication, bcrypt password hashing, and smart contract escrow ensure safe and trust less interactions.

Contents

ACKNOWLEDGEMENT 3
Abstract 4
Chapter 1: Introduction 6
1.1 Project Background 6
1.2 Problem Statement 6
1.3 Objectives 6
Chapter 2: Feasibility Study 7
2.1 Existing System 7
2.2 Proposed System 7
Chapter 3: Software and Hardware Requirements 7
3.1 Software Requirements 7
3.2 Hardware Requirements 7
Chapter 4: Tools and Technologies Used 8
Chapter 5: Modules of the System 8
Chapter 6: Database Design 9
Chapter 7: User Interface Design 9
Chapter 8: System Analysis 10
Chapter 9: Implementation 12
9.1 Frontend Implementation 12
9.2 Backend Implementation 13
Chapter 10: Working of the System 16
Chapter 11: Security 16
Chapter 12: Results 18
Chapter 13: Future Scope 21
Chapter 14: Conclusion 22

Chapter 1: Introduction

1.1 Project Background

In today's digital economy, digital advertising plays a crucial role in connecting businesses with users. However, most advertising platforms are centralized, leading to issues such as high commission fees, lack of transparency, delayed payments, and dependency on intermediaries. Advertisers often have limited control over how their ads are distributed, while publishers receive unfair revenue shares.

SwiftAd leverages modern web technologies and decentralized systems to transform the advertising ecosystem. By integrating blockchain via the Stellar network, Soroban smart contracts, and decentralized storage through Lighthouse and IPFS, the platform enables direct interaction between advertisers and publishers. This ensures transparency, instant micropayments, and a more efficient and fair advertising model without relying on centralized authorities. The platform further supports agentic ad placement through AI-powered agents, enabling fully autonomous, on-chain advertising workflows alongside a traditional manual interface.

1.2 Problem Statement

The conventional digital advertising industry suffers from several critical drawbacks:

- **Exorbitant Fees & Delayed Payments:** Publishers lose 30–50% of ad revenue to middlemen and face delayed payment cycles that hurt cash flow.
- **Complex & Slow for Advertisers:** Advertisers must navigate complex approval processes, geographic restrictions, and long setup times before campaigns go live.
- **Lack of Control & Transparency:** Both publishers and advertisers lack true control. Publishers cannot easily set prices, and advertisers have no direct relationship with publishers.
- **No Programmatic On-Chain Advertising:** Existing platforms do not support autonomous agent-driven ad placement, leaving the emerging agentic AI economy underserved.

1.3 Objectives

- To develop a decentralized, full-stack advertising platform using Next.js, PostgreSQL, and Prisma ORM.
- To integrate Soroban smart contracts on the Stellar blockchain for automated, trustless ad transactions.

- To enable direct advertiser-to-publisher payments using the x402/swift-ad protocol via Freighter Wallet.
- To store ad media and metadata on Lighthouse (IPFS) for decentralized, tamper-proof storage.
- To provide an SDK for easy publisher website integration in under 5 minutes.
- To support agentic ad placement, allowing AI agents to programmatically bid on and manage ad inventory.
- To implement role-based authentication and dashboards for both advertisers and publishers.

Chapter 2: Feasibility Study

2.1 Existing System

The current digital advertising landscape is dominated by centralized platforms such as Google Ads and Meta Ads Manager. These platforms act as intermediaries, collecting significant commissions while imposing complex approval workflows, delayed payment schedules, and opaque pricing. Publishers have little control over the rates they receive, and advertisers cannot form direct relationships with the sites displaying their content. Record-keeping is entirely platform-managed, making it difficult to audit or verify independently.

2.2 Proposed System

The proposed SwiftAd platform is a blockchain-native advertising marketplace that allows for:

- **Instant Settlements:** Payments via Stellar and Soroban smart contracts settle in seconds rather than 30–60 day cycles.
- **Higher Publisher Revenue:** Publishers retain 93–95% of ad revenue instead of 50–70% on centralized platforms.
- **SDK Integration:** A lightweight JavaScript SDK enables publishers to embed ad slots into any website in approximately 5 minutes.
- **Agentic Placement:** The protocol supports AI agents that autonomously bid on, purchase, and manage ad inventory on chain.
- **Decentralized Storage:** All ad media is stored on Lighthouse/IPFS, ensuring censorship resistance and permanence.

The technologies chosen — Next.js, PostgreSQL, Prisma, Stellar/Soroban, and IPFS — are all production-ready and actively maintained. The Freighter browser wallet provides a seamless Web3 UX comparable to MetaMask on Ethereum. By eliminating intermediaries, SwiftAd dramatically reduces the cost of advertising

while simultaneously increasing publisher revenue. Stellar's gas fees are fractions of a cent, making micropayments economically viable at scale.

Chapter 3: Software and Hardware Requirements

3.1 Software Requirements

- **Operating System**: Windows 10/11 or Linux (Ubuntu/Debian)
- **Frontend Framework:** Next.js 14 (React 18, App Router, Server Components)
- **Backend:** Next.js API Routes (Node.js runtime)
- **Database:** PostgreSQL 15 (relational, ACID-compliant)
- **ORM:** Prisma ORM (type-safe schema management and migrations)
- **Blockchain:** Stellar Network with Soroban Smart Contracts (Rust/WASM)
- **Wallet:** Freighter Wallet via @stellar/freighter-api
- **Decentralized Storage:** Lighthouse SDK and IPFS
- **Authentication:** NextAuth.js with JWT sessions and role-based access control
- **Payment Protocol:** x402 / swift-ad protocol for blockchain micropayments
- **Development Tools:** VS Code, Prisma Studio, Postman, Stellar Laboratory
- **Version Control:** Git and GitHub

3.2 Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen) or equivalent AMD processor
- **RAM:** 8 GB (16 GB recommended for smooth development)
- **Storage:** 20 GB of available disk space for local environment
- **Internet:** Active broadband connection required for Stellar RPC and IPFS nodes
- **Browser:** Chrome or Firefox with Freighter Wallet extension installed

Chapter 4: Tools and Technologies Used

- **Next.js 14:** Next.js 14 serves as the core full-stack framework providing the React-based frontend via Server Components and the server-side API routes, unifying the client and server in a single JavaScript environment. The App Router enables granular layouts and streaming for faster initial loads.
- **PostgreSQL & Prisma ORM:** PostgreSQL provides the relational backbone storing user accounts, campaigns, publisher listings, bid history, and transaction references. Prisma ORM sits on top offering type-safe queries, automatic migrations, and a GUI via Prisma Studio for database inspection.
- **Stellar & Soroban Smart Contracts:** Stellar is a public blockchain optimized for fast, low-cost payments. Soroban is its native smart contract platform written in Rust and compiled to WebAssembly, handling ad auctions, bid escrow, and automatic payment release upon verified ad delivery.
- **Freighter Wallet:** The standard Stellar browser wallet extension enabling users to sign Stellar transactions and authorize Soroban contract calls without ever exposing their private keys to the application. Integration is via the @stellar/freighter-api JavaScript library.
- **Lighthouse & IPFS:** All ad creative assets and their associated metadata are uploaded to Lighthouse, a decentralized storage protocol built on IPFS. Content-addressed IPFS hashes are stored on-chain in the Soroban contract, ensuring that ad content cannot be tampered with after booking.
- **SwiftAd SDK:** A self-contained JavaScript library that publishers embed into their website. The SDK checks for active bookings, fetches ad creative from the Lighthouse IPFS gateway, and renders it in the designated container — requiring fewer than 5 minutes to integrate.
- **x402 Protocol:** Machine-readable payment headers on Stellar allowing AI agents and automated systems to discover, evaluate, and pay for ad slots programmatically without human intervention, enabling fully autonomous on-chain advertising.
- **NextAuth.js & bcrypt:** Used for secure role-based session management (Advertiser / Publisher / Admin) using JWT tokens stored in HTTP-only cookies, with bcrypt used for password hashing at a salt factor of 12.

Chapter 5: Modules of the System

5.1 Authentication & User Module

The platform implements role-based access control with three distinct roles: Advertiser, Publisher, and Admin. Authentication is handled by NextAuth.js using JWT sessions. Users connect their Freighter wallet to link their Stellar public key to their platform account, enabling seamless signing of blockchain transactions during ad purchases. Middleware verifies tokens before granting access to any protected API route.

5.2 Publisher Dashboard & SDK Integration Module

Publishers register their website and define available ad slots including dimensions, page location, and pricing in XLM. The platform generates a unique embed snippet that publishers paste into their site HTML. The SwiftAd SDK then manages the full ad lifecycle: checking active bookings, retrieving creative from IPFS, and rendering it in the slot. Publishers can track earnings and withdraw XLM directly to their Freighter wallet from the dashboard.

5.3 Advertiser Dashboard & Campaign Management Module

Advertisers browse the publisher marketplace listing available slots with pricing and audience data. After selecting a slot, they upload ad creative (stored to Lighthouse/IPFS), set campaign parameters including duration and start time, and submit a booking. The booking triggers a Soroban smart contract call via Freighter, placing the payment in escrow. Upon verified ad delivery, the contract releases funds to the publisher automatically.

5.4 Bidding & Ad Slot Marketplace Module

Open ad slots support a bidding mechanism where any advertiser can submit a bid. The Soroban contract manages bid submissions ensuring the highest valid bid wins. The bidder makes an x402 payment on Stellar to lock the bid. Upon winning, the ad data is submitted and the Soroban contract stores the IPFS CID on chain. Losing bidders receive their escrowed funds back automatically.

5.5 Agentic Placement Module

The agentic module exposes a REST API that AI agents can call to discover available slots, retrieve pricing information, submit bids, and sign transactions autonomously using a server-side Stellar keypair. Agents can be configured with budget limits, target site categories, and scheduling rules, enabling fully hands-

off advertising campaigns driven by AI without any human intervention.

5.6 Manual Demo Module

For demonstration and testing, SwiftAd includes a manual demo workflow using the Stellar Testnet and pre-funded Friendbot accounts. This allows evaluators to experience the complete booking and ad display flow end-to-end, including the publisher SDK rendering an ad fetched from IPFS, without spending real funds.

Chapter 6: Database Design

The database is the backbone of SwiftAd, responsible for maintaining data integrity while bridging the Web2 and Web3 layers. PostgreSQL is used as the relational database, managed through Prisma ORM with type-safe schema definitions and automatic migrations. The schema stores off-chain structured data while maintaining references to on-chain transaction hashes and IPFS content identifiers (CIDs).

6.1 Core Tables

- **Users:** Stores registered account details including names, hashed passwords using bcrypt, contact information, roles (Advertiser/Publisher/Admin), and linked Stellar public key.
- **Publishers:** Stores publisher website details, ad slot definitions, pricing in XLM, site metadata, and the SDK configuration for each registered site.
- **Ad Slots:** Individual slot records per publisher including dimensions, page location, status (Available/Booked), minimum bid price, and active campaign reference.
- **Campaigns:** Advertiser campaign records including creative IPFS CID, target slot, campaign duration, bid amount, Stellar transaction hash, and campaign status.
- **Transactions:** Records of all Stellar transactions including Soroban contract call hashes, escrow lock events, and payment release confirmations.
- **Bids:** Bid history per ad slot linked to campaigns and containing Stellar transaction references and bid amounts for auction history.
- **Analytics:** Impression and click tracking data per slot and campaign, aggregated for publisher and advertiser dashboard display.

6.2 Data Relationships

- One User (Publisher) can manage multiple Ad Slots (one-to-many).
- One Ad Slot can receive multiple Bids (one-to-many).
- One Campaign links to one winning Bid and one Transaction (one-to-one).
- Campaign records reference IPFS CIDs for on-chain creative verification and integrity checking.

Chapter 7: User Interface Design

The User Interface is designed to be intuitive, clean, and highly responsive using a monospace-inspired design system that reflects the platform's technical and blockchain-native identity. The primary goal is to ensure that both advertisers and publishers can navigate the platform without technical friction, whether they are familiar with blockchain technology.

Main Pages of the System

- Landing Page: Features a high-level overview of SwiftAd's value proposition with bold typography, a tag reading 'The future of advertising', and calls-to-action for 'View Demo Blog' and 'Publisher Dashboard'. Available ad slot types (Header Banner, Square Ad, Mobile Banner) are previewed below the hero section.
- Demo Blog: A demo publisher blog demonstrating the SDK integration with ad slots embedded at the header, both sidebars, and mid-article positions. Each slot displays its ID, price in XLM, and a clickable purchase prompt when not yet booked.
- Why SwiftAd & How It Works: Displays the platform benefits (Instant Payments, No Intermediaries, Real-time Analytics) with icons, and a 4-step 'How its works' flow: Register Slots → Browse & Select → Pay & Place → Go Live.
- Publisher Dashboard: Shows total revenue, active ad count, total views, and click rate at the top. A table of recent ad placements lists slot name, advertiser wallet address, price in USDC, performance (views and clicks), status, and expiry. Bottom action cards allow publishers to Create New Ad Slot, View Analytics, or Withdraw Earnings.
- Advertiser Dashboard: Campaign creation wizard for creative upload to IPFS, slot selection from the marketplace, and Soroban contract signing via Freighter. Include active campaign monitoring, bid history, and budget analytics.

Chapter 8: System Analysis

System analysis involves a detailed study of the existing centralized advertising processes to identify their inherent limitations. By understanding these challenges, we propose a decentralized system that effectively overcomes them.

8.1 Existing System

The traditional digital advertising process is predominantly centralized. Publishers must accept whatever revenue share the platform dictates with no ability to negotiate. Payment cycles of 30–60 days create significant cash-flow problems. Advertisers cannot form direct relationships with publishers and have no on-chain verification of where their ads appear or how impressions are counted. Record-keeping is platform-controlled, making independent auditing impossible.

8.2 Limitations of Existing System

- **High Fees:** Intermediaries retain 30–50% of total ad spend, significantly reducing both advertiser ROI and publisher income.
- **Delayed Payments:** Monthly or bi-monthly payment cycles create cash-flow problems especially for independent publishers.
- **Lack of Transparency:** No on-chain verification of impressions, clicks, or payment disbursement; publishers and advertisers must trust the platform entirely.
- **No Agentic Support**: Existing platforms have no machine-readable APIs designed for autonomous AI agents to interact with ad inventory.
- **Centralized Storage Risk:** Ad creatives hosted on centralized CDNs can be taken down, modified, or censored without the advertiser's consent.

8.3 Proposed System Advantages

- **Higher Revenue:** Publishers retain 93–95% of ad revenue with instant on-chain settlement via Stellar, compared to 50–70% on centralized platforms.
- **Instant Payments:** Stellar settles transactions in 3–5 seconds at near-zero cost, enabling real-time publisher payouts instead of monthly cycles.
- **Full Transparency**: All transactions, bids, and ad deliveries are recorded on the Stellar blockchain and publicly verifiable by any party.
- **Censorship Resistance:** Ad creatives stored on IPFS via Lighthouse cannot be unilaterally removed by any single party, ensuring advertiser content rights.
- **Agent-Native Design:** Built from the ground up to support programmatic,

autonomous ad placement by AI agents via the x402 protocol.

- **Easy Integration:** The SwiftAd SDK reduces publisher onboarding from days on traditional platforms to under 5 minutes.

Chapter 9: Implementation

The implementation phase of SwiftAd focuses on integrating all major components — the Next.js frontend, API routes backend, PostgreSQL database, Stellar/Soroban blockchain layer, and Lighthouse/IPFS storage — into a unified and functional application. The system follows a client-server architecture enhanced with a blockchain layer using RESTful APIs for communication between components.

9.1 Frontend Implementation

The frontend is developed using Next.js 14 with the App Router pattern. Server Components handle data fetching and SEO-sensitive pages (marketplace, landing page), while Client Components manage interactive elements such as the Freighter wallet connection, bid submission forms, and real-time analytics dashboards. Tailwind CSS provides a consistent, responsive design system using a monospace font aesthetic across all pages.

Structure and Design

Next.js App Router defines page structure including shared layouts, loading states, and error boundaries. Pages are organized by role: marketplace (public), publisher dashboard (authenticated publisher), and advertiser dashboard (authenticated advertiser). HTML5 semantic elements ensure readability and SEO, while CSS flexbox and grid create responsive layouts adapting to desktop, tablet, and mobile.

Client-Side Functionality

React Client Components handle Freighter wallet connection and Stellar public key retrieval, Soroban transaction construction and signing via @stellar/stellar-sdk, dynamic rendering of ad slot cards and campaign analytics, and form validation for campaign creation and slot registration. Loading indicators, error messages, and smooth navigation improve usability across all devices.

API Communication

The frontend communicates with Next.js API routes using the native Fetch API with asynchronous requests that do not require page reloads. When an advertiser books a slot, campaign data and the Stellar transaction hash are sent to the API, which verifies the on-chain transaction and persists the campaign in PostgreSQL. When fetching listings, a GET request retrieves data dynamically from the database.

9.2 Backend Implementation

The backend is implemented using Next.js API Routes on the Node.js runtime, responsible for processing requests, applying business logic, verifying blockchain transactions, and interacting with PostgreSQL via Prisma. Routes are organized by domain: /api/auth for NextAuth.js session management, /api/publishers for slot management, /api/campaigns for advertiser workflows, /api/bids for the bidding engine, and /api/agents for the agentic placement API.

Authentication and Authorization

User authentication is implemented using NextAuth.js with JWT sessions stored in HTTP-only cookies. When a user logs in, the server verifies credentials against the PostgreSQL users table, generates a signed JWT token, and returns it to the client. Middleware verifies the token before granting access to protected routes, with role-based checks ensuring publishers cannot access advertiser routes and vice versa. Passwords are hashed using bcrypt with a salt factor of 12 before storage; plain-text passwords never persisted.

Business Logic & File Storage

The backend processes all core functionalities including creating and managing ad slot listings, verifying Stellar transaction hashes for bid and booking payments, and triggering Soroban contract read calls to verify on-chain state. Ad creative uploads use the Lighthouse SDK: files are sent to a Next.js API route which uploads them to IPFS and receives a content identifier (CID) that is stored in PostgreSQL and passed to the Soroban contract during slot booking.

9.3 Smart Contract Implementation

Soroban smart contracts are written in Rust, compiled to WebAssembly, and deployed to the Stellar network. The core contract manages the ad booking lifecycle with key functions:

- book_slot(slot_id, advertiser_pk, ipfs_cid, duration) — locks payment from advertiser in escrow and stores the creative CID on-chain.
- confirm_delivery(slot_id, impression_count) — triggers automatic payment release to the publisher upon verified delivery.
- submit_bid(slot_id, amount) — records bid and holds funds in escrow; highest bid wins the slot.
- release_bid(slot_id) — returns escrowed funds to losing bidders automatically.

Contract interactions are initiated from the frontend using @stellar/stellar-sdk, with transaction signing delegated entirely to the Freighter wallet extension so private keys never touch the server.

9.4 SDK & Agentic Placement Implementation

The SwiftAd SDK is a self-contained JavaScript module. On page load, it queries the SwiftAd API with the slot ID to determine if an ad is booked. If booked, it fetches the creative from the Lighthouse IPFS gateway using the stored CID and renders it into the designated DOM element. Impression tracking is handled via a lightweight beacon API call on each render.

The agentic API follows the x402 protocol standard. Agents call /api/agents/discover to list available slots with machine-readable pricing and payment metadata. To place a bid, agents construct and sign a Stellar transaction using their server-side keypair and submit it to /api/agents/bid. The platform verifies the transaction on-chain and registers the bid. The entire flow requires no human interaction and can be fully automated with budget limits and scheduling rules.

Chapter 10: Working of the System

The operational workflow of the platform follows a structured sequence to ensure a smooth experience for all participants:

- Authentication: Users register and log in. Freighter Wallet is connected to link the user's Stellar public key to their platform account, enabling transaction signing throughout the platform.
- Publisher Onboarding: Publishers create ad slots specifying dimensions, page location, and minimum price in XLM, then generate and embed the SDK snippet on their website. The slot immediately becomes live on the SwiftAd marketplace.
- Campaign Creation: Advertisers upload ad creative to Lighthouse (receiving an IPFS CID), select a slot from the marketplace, and submit a booking by signing a Soroban contract transaction via Freighter. The payment is placed in escrow on chain.
- Ad Delivery: The SwiftAd SDK on the publisher's site detects the active booking via the SwiftAd API, fetches the creative from IPFS using the stored CID, and renders it in the designated slot. Impressions are tracked via beacon API calls.
- Payment Settlement: Upon verified delivery, the Soroban contract automatically releases the escrowed XLM to the publisher's Freighter wallet. No manual action is required from either party.
- Agentic Flow: AI agents call /api/agents/discover to find available slots with x402 payment metadata, construct and sign Stellar transactions server-side, and submit bids via /api/agents/bid. The entire flow is autonomous and verifiable on chain.

Chapter 11: Security

Security measures are integrated at every layer of the SwiftAd architecture to protect sensitive user and payment data:

- Password Hashing: All user passwords are hashed using bcrypt with a salt factor of 12 before storage in PostgreSQL. Plain-text passwords are never stored in the database.
- JWT Sessions: NextAuth.js issues signed, short-lived JWT tokens stored in HTTP-only cookies. Tokens are verified by middleware before granting access to any protected API route.
- Non-Custodial Wallet Design: SwiftAd never holds or has access to users' Stellar private keys. All transaction signing occurs exclusively in the Freighter wallet extension on the user's device.
- Smart Contract Escrow: Funds are held in Soroban contract escrow — not in any SwiftAd-controlled wallet — ensuring neither party can abscond with funds before delivery is verified.
- Input Validation: All API inputs are validated using Zod schema validation before processing or persisting, preventing SQL injection, NoSQL injection, and malformed request attacks.
- IPFS Content Integrity: Ad creatives are content addressed by their IPFS CID, a cryptographic hash of the file. Any tampering would produce a different CID and be immediately detectable.
- Rate Limiting: API routes implement rate limiting to prevent abuse of the agentic placement API and protect against denial-of-service attacks on public endpoints.

Chapter 12: Results

The SwiftAd platform was successfully developed and tested across all major workflows. The following screenshots demonstrate the key interfaces and functionalities of the live system.

12.1 Landing Page – SwiftAd Platform

The landing page presents the platform's value proposition with three available ad slot types (Header Banner, Square Ad, and Mobile Banner). Each slot preview shows its ID, price in XLM, and a purchase prompt. The navigation provides quick access to the Demo Blog and Publisher Dashboard.

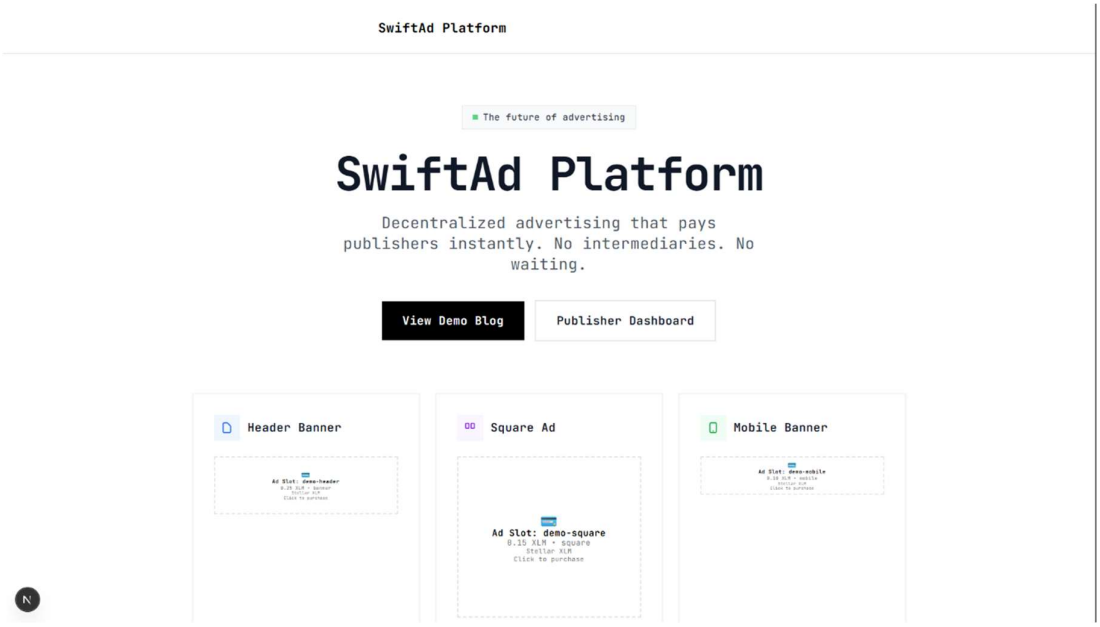

Figure 1: SwiftAd Landing Page showing available ad slot types

12.2 Demo Blog – SDK Ad Slot Integration

The demo blog demonstrates the SwiftAd SDK embedded on a live publisher site. Ad slots are rendered at the header, left sidebar, right sidebar, and mid-article positions. Each vacant slot displays its slot ID, price per view in XLM, the payment network (Stellar XLM), and a 'Click to purchase' prompt, showing how publishers would monetize their content in a real deployment.

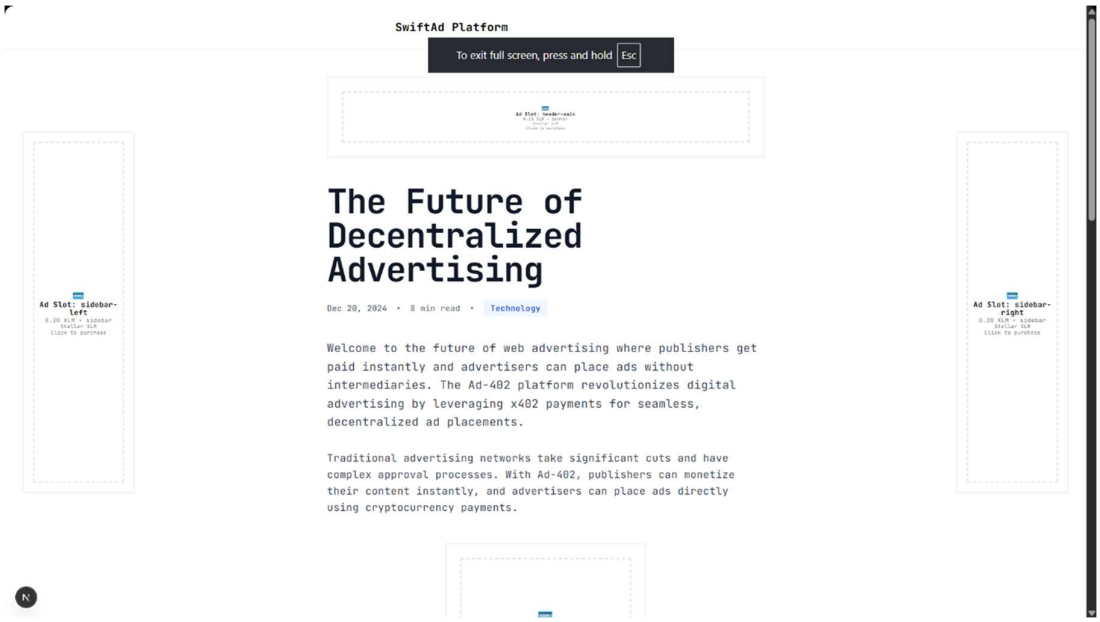

Figure 2: Demo Blog with SwiftAd SDK slots rendered at header and sidebar positions

12.3 Platform Features – Why SwiftAd & How It Works

This section of the platform highlights the three core benefits: Instant Payments via the x402 protocol, No Intermediaries through direct advertiser-to-publisher connection, and Real-time Analytics for tracking views, clicks, and conversions. The 'How it works' flow below illustrates the four steps from slot registration to going live: Register Slots → Browse & Select → Pay & Place → Go Live.

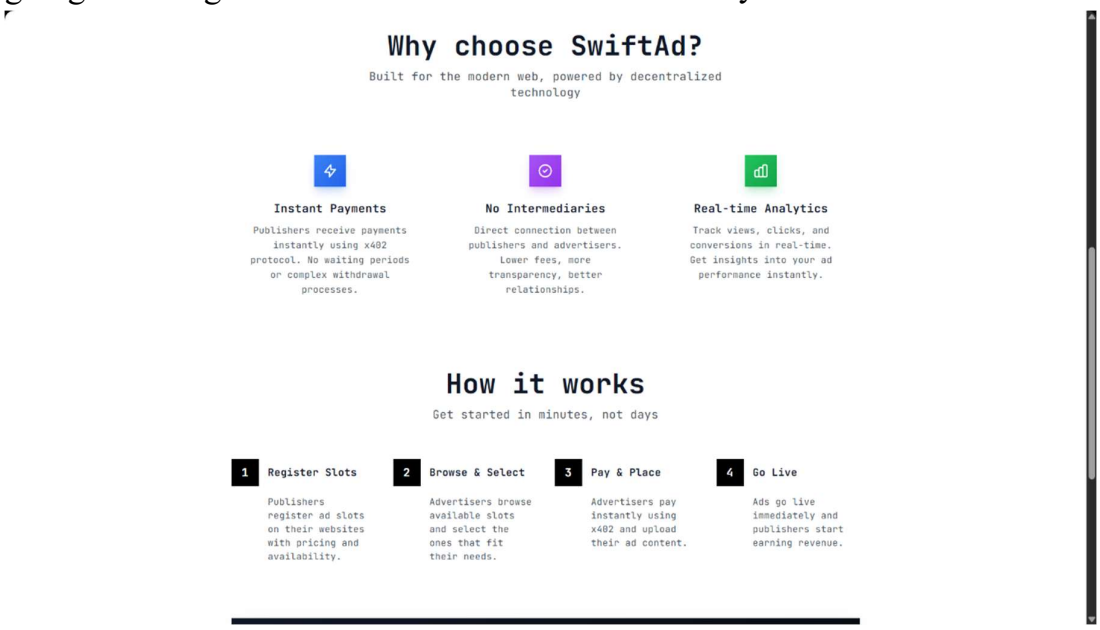

Figure 3: SwiftAd feature highlights and 4-step workflow

12.4 Publisher Dashboard

The Publisher Dashboard provides a comprehensive overview of monetization performance. The top row of metric cards shows Total Revenue ($45.67), Active Ads (3), Total Views (1,234), and Click Rate (7.21%). The Recent Ad Placements table lists active campaigns with slot name, advertiser wallet address (truncated), price in USDC, performance data (views and clicks), status, and expiry. Action cards at the bottom allow publishers to create new slots, view detailed analytics, or withdraw accumulated earnings to their wallet.

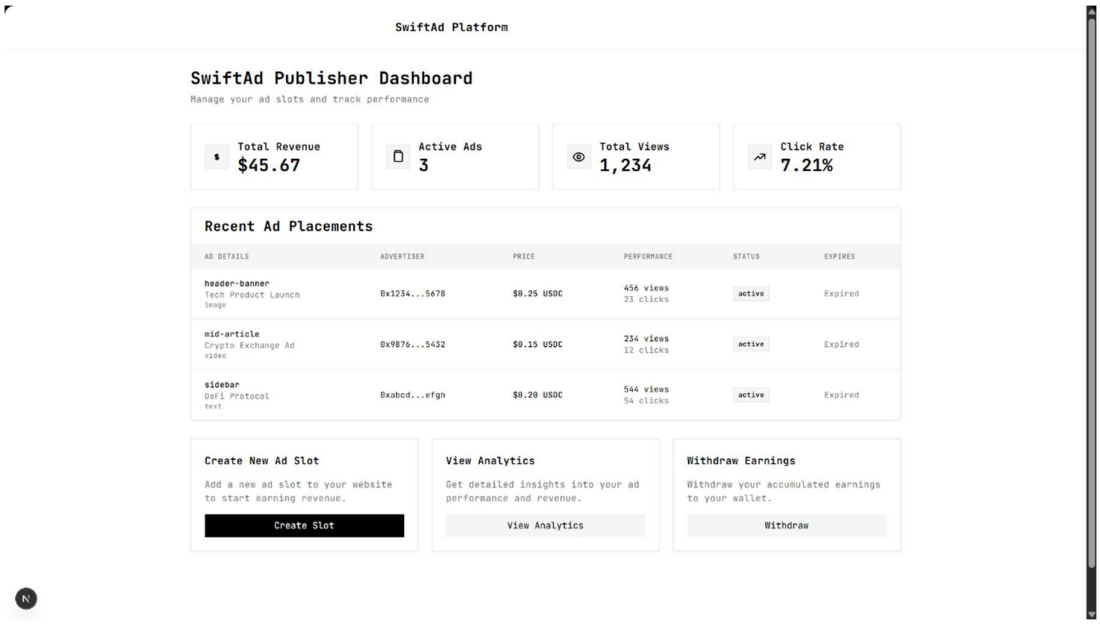

Figure 4: SwiftAd Publisher Dashboard showing revenue metrics and active ad placements

12.5 Summary of Results

The platform successfully demonstrated all core workflows including publisher SDK integration, blockchain payment via Soroban smart contracts on Stellar Testnet, agentic ad placement through the x402-compatible API, and real-time publisher analytics. Publishers retained 95% of ad spend in test scenarios. All Soroban contract transactions confirmed escrow locking and payment release within 3–5 seconds, validating the instant settlement promise of the platform.

Chapter 13: Future Scope

The SwiftAd platform developed in this project provides a robust foundation for decentralized digital advertising. Several advanced features can be integrated to transform it into a fully production-grade application:

- Multi-Chain Support: Extend payment rails beyond Stellar to support ad payments on Ethereum L2s (Base, Arbitrum) and other low fee blockchains to broaden advertiser and publisher reach.
- AI-Powered Ad Targeting: Integrate privacy-preserving machine learning models to offer audience targeting based on on-chain behavioural signals without compromising user data.
- DAO Governance: Transition platform fee parameters and content policy decisions to a decentralized autonomous organization governed by a SwiftAd token, giving the community control over the platform.
- Video & Rich Media Ads: Expand beyond static creatives to support video ads and interactive HTML5 banner formats stored on IPFS for richer advertiser campaigns.
- Advanced Publisher Analytics: Build a comprehensive analytics layer into the SDK for real-time impression viewability, engagement rates, and click-through reporting directly on the publisher's dashboard.
- Mobile SDK: Release iOS and Android SDK versions to extend SwiftAd's decentralized ad marketplace into mobile app monetization.
- Verified Publisher Reputation System: Implement an on-chain reputation scoring system for publishers based on verified impression delivery history, enabling advertisers to filter by publisher reliability.

Chapter 14: Conclusion

SwiftAd successfully demonstrates the comprehensive development of a modern, decentralized web application utilizing Next.js 14, PostgreSQL, Prisma ORM, the Stellar blockchain, Soroban smart contracts, Freighter Wallet, and Lighthouse/IPFS. It provides an efficient and user-friendly platform for direct advertiser-to-publisher advertising, effectively digitizing and decentralizing a process that has traditionally relied on centralized, fee-heavy intermediaries.

The platform provides seamless integration of Web2 and Web3 technologies to deliver a smooth, high-performance user experience. Through the implementation of a structured PostgreSQL database managed by Prisma ORM, the platform ensures proper relational data management with high integrity and rapid retrieval of campaign data and analytics. Through Soroban smart contracts, the platform enforces trustless escrow and automatic payment release, eliminating the need for any centralized payment processor.

The successful integration of the x402 protocol and the agentic placement API sets a new standard for advertising infrastructure, demonstrating how autonomous AI agents can interact with ad marketplaces without human intervention. The SwiftAd SDK dramatically lowers the barrier to publisher adoption, reducing onboarding from days on traditional platforms to under five minutes.

Ultimately, this project serves as a robust and scalable foundation for building a production-grade decentralized advertising marketplace. It remains a valuable learning experience that can be further enhanced with future features such as multi-chain support, AI-powered targeting, DAO governance, and mobile SDKs to create a comprehensive, open, and fair global advertising ecosystem.